

Ideasson Technology Solutions

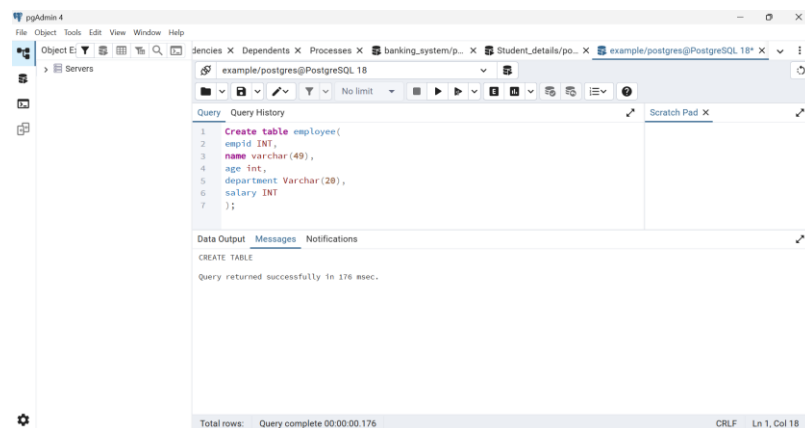
Name: Durga R

Domain: Springboot

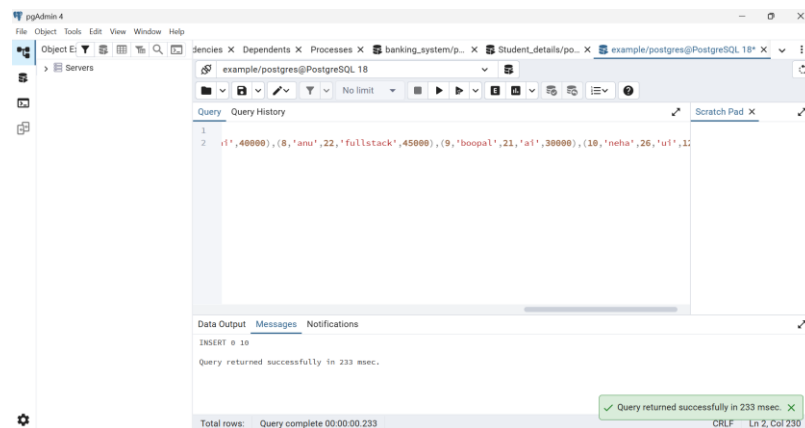
SQL Coding Practice

1. Employee Table (CRUD Operations)*

1.Create an *Employee* table.



2. Insert at least *10 employee records*.



3.Display all records.

The screenshot shows the pgAdmin 4 interface with a query window open. The query is a SELECT statement that retrieves all columns from the employee table. The results are displayed in a table with 10 rows and 5 columns: empid, name, age, department, and salary.

empid	name	age	department	salary
1	sue	20	sales	20000
2	archu	23	product	22000
3	jan	20	testing	25000
4	so	23	sales	22000
5	kalai	19	ui	80000
6	srinidhi	28	analytics	50000
7	ram	25	ai	40000
8	anu	22	fullstack	45000
9	boopal	21	ai	30000
10	neha	26	ui	12000

Total rows: 10 Query complete 00:00:00.306 CRLF Ln 1, Col 24

4.Update an employee's salary.

The screenshot shows the pgAdmin 4 interface with a query window open. The query is an UPDATE statement that sets the salary of the employee with empid 6 to 78000. The Data Output tab shows the successful execution of the query.

```

1 update employee
2 set salary = 78000
3 where empid = 6;

```

UPDATE 1
Query returned successfully in 120 msec.

Total rows: Query complete 00:00:00.120 CRLF Ln 3, Col 17

The screenshot shows the pgAdmin 4 interface with a query window open. The query is a SELECT statement that retrieves all columns from the employee table. The results are displayed in a table with 10 rows and 5 columns: empid, name, age, department, and salary. The salary for empid 6 has been updated to 78000.

empid	name	age	department	salary
1	sue	20	sales	20000
2	archu	23	product	22000
3	jan	20	testing	25000
4	so	23	sales	22000
5	kalai	19	ui	80000
6	srinidhi	28	analytics	78000
7	ram	25	ai	40000
8	anu	22	fullstack	45000
9	boopal	21	ai	30000
10	neha	26	ui	12000

Total rows: 10 Query complete 00:00:00.145 CRLF Ln 1, Col 24

5.Delete one employee record.

The screenshot shows the pgAdmin 4 interface with a query window open. The query is a DELETE statement that deletes the employee record with empid 5. The Data Output tab shows the successful execution of the query.

```

1 delete from employee
2 where empid = 5;

```

DELETE 1
Query returned successfully in 115 msec.

Total rows: Query complete 00:00:00.115 CRLF Ln 2, Col 17

pgAdmin 4 interface showing a query result for 'select * from employee;'. The table has 9 rows and 6 columns: empid, name, age, department, salary. The data is as follows:

empid	name	age	department	salary
1	swe	20	sales	20000
2	archu	23	product	22000
3	jan	20	testing	20000
4	so	23	sales	22000
5	ram	25	ai	40000
6	anu	22	fullstack	45000
7	boopal	21	ai	30000
8	neha	26	ai	12000
9	sridhar	28	analytics	70000

Total rows: 9 Query complete 00:00:00.133

6.Drop the table.

pgAdmin 4 interface showing the successful execution of 'drop table employee;'. The message pane displays 'DROP TABLE' and 'Query returned successfully in 186 msec.'.

Total rows: Query complete 00:00:00.186

pgAdmin 4 interface showing an error message for 'select * from employee;'. The message pane displays 'ERROR: relation "employee" does not exist' and 'LINE 1: select * from employee;'.

Total rows: Query complete 00:00:00.121

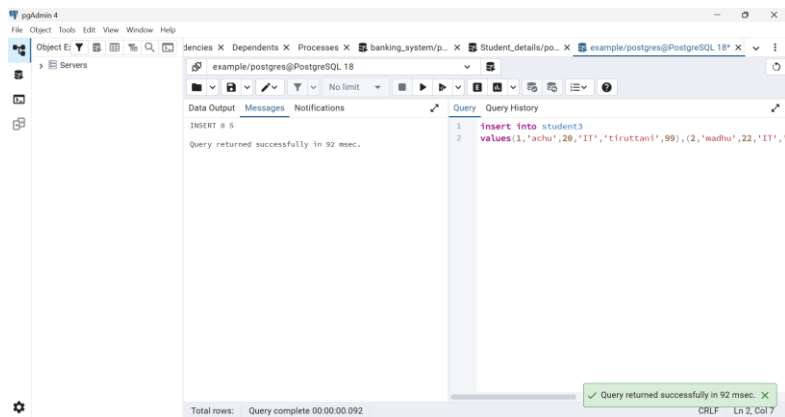
2. Student Table*

1. Create a *Student* table.

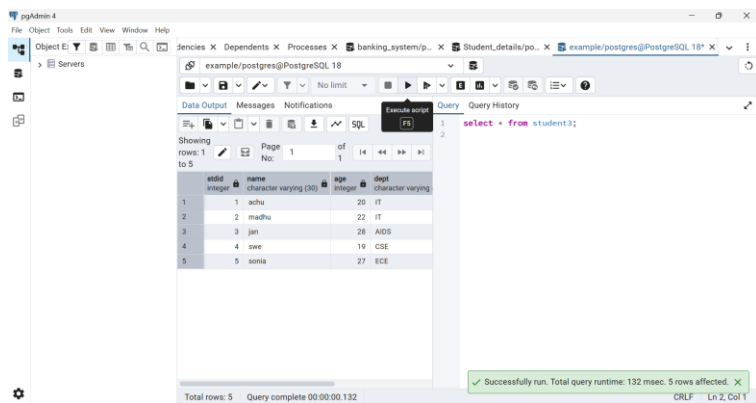
pgAdmin 4 interface showing the successful execution of 'create table student3;'. The message pane displays 'CREATE TABLE' and 'Query returned successfully in 88 msec.'.

Total rows: Query complete 00:00:00.088

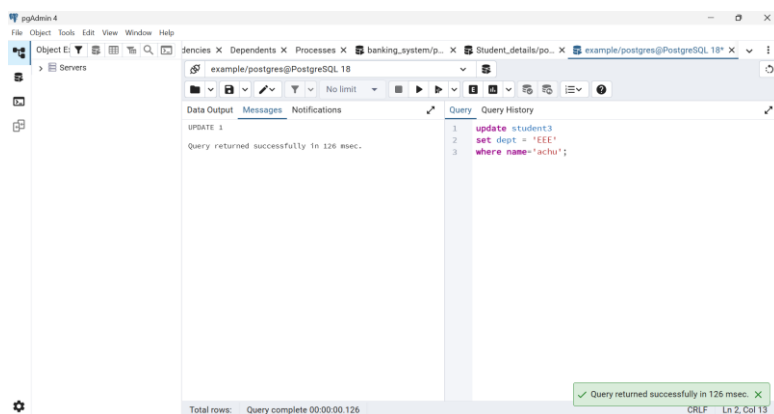
2.Insert at least *10 student records*.

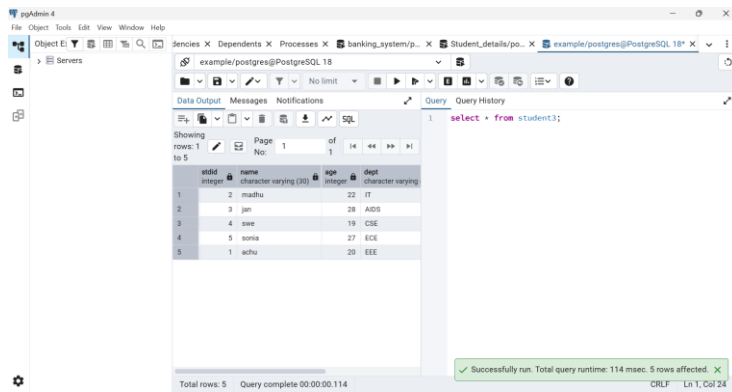


3.Display all students.

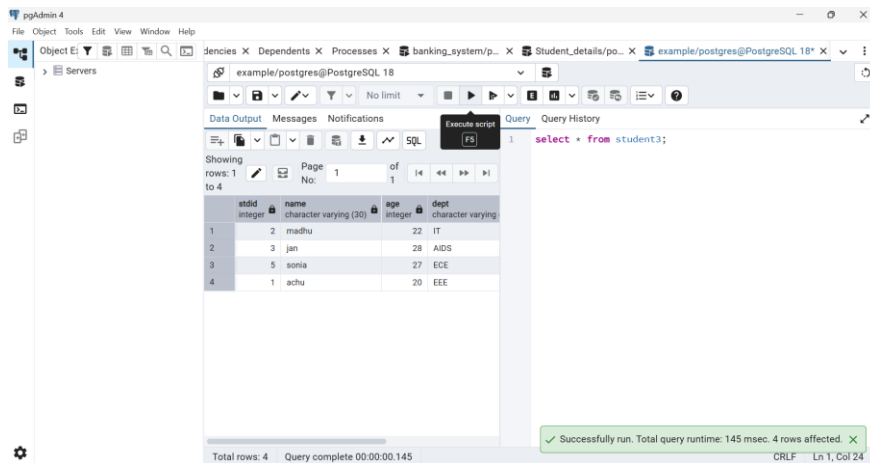
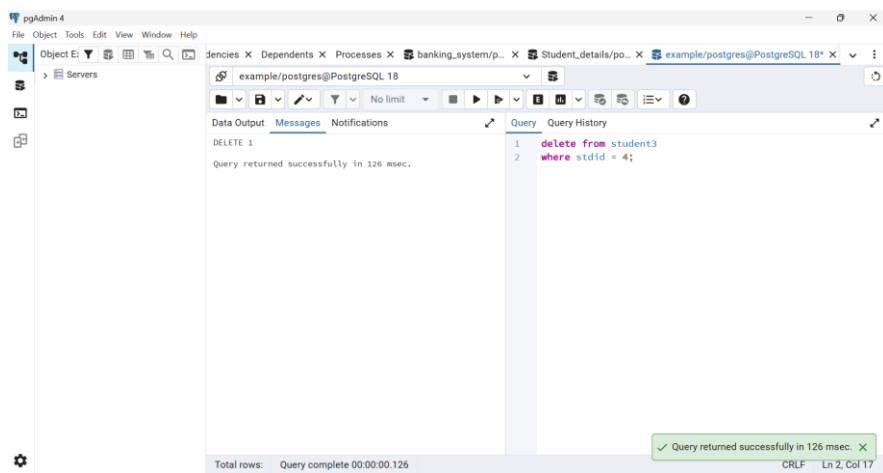


4.Update one student's department.





5.Delete one student record.



6.Sort students by name.

The screenshot shows the pgAdmin 4 interface with a SQL query window. The query is: `select * from student3 order by age desc`. The Data Output tab shows the results of the query, which are sorted by age in descending order. The status bar at the bottom indicates: "Total rows: 4 Query complete 00:00:00.155" and "Successfully run. Total query runtime: 155 msec. 4 rows affected."

stid	name	age	dept
3	jan	28	ADS
5	sonia	27	ECE
2	madhu	22	IT
1	achu	20	EEE

The screenshot shows the pgAdmin 4 interface with a SQL query window. The query is: `select * from student3 order by age asc`. The Data Output tab shows the results of the query, which are sorted by age in ascending order. The status bar at the bottom indicates: "Total rows: 4 Query complete 00:00:00.168" and "Successfully run. Total query runtime: 168 msec. 4 rows affected."

stid	name	age	dept
1	achu	20	EEE
2	madhu	22	IT
5	sonia	27	ECE
3	jan	28	ADS

3. SQL Filtering

The screenshot shows the pgAdmin 4 interface with a SQL query window. The query is: `CREATE TABLE Employee ( emp_id INT PRIMARY KEY, emp_name VARCHAR(50), department VARCHAR(50), salary DECIMAL(10,2), city VARCHAR(50) );`. The Data Output tab shows the message: "CREATE TABLE Query returned successfully in 124 msec." The status bar at the bottom indicates: "Total rows: Query complete 00:00:00.124" and "Query returned successfully in 124 msec."

WHERE:

select * from Employee

where department = 'IT';

pgAdmin 4

example/postgres@PostgreSQL 18

SQL

Showing rows: 1 to 4 of 1

emp_id [PK] integer	emp_name character varying (50)	department character varying (50)	salary numeric (10,2)	city character varying (50)
1	101 Arun	IT	50000.00	Chennai
2	103 Karthik	IT	60000.00	Chennai
3	106 Sneha	IT	70000.00	Bangalore
4	110 Meena	IT	75000.00	Bangalore

Query

```
1 select * from Employee
2 where department = 'IT';
```

Successfully run. Total query runtime: 168 msec. 4 rows affected.

Total rows: 4 Query complete 00:00:00.168 CRLF Ln 2, Col 21

LIKE:

SELECT * FROM Employee

WHERE emp_name LIKE 'Ra%';

And ('A__')

SELECT * FROM Employee

WHERE emp_name LIKE 'R____';

And

SELECT * FROM Employee

WHERE emp_name LIKE '_a____';

pgAdmin 4

example/postgres@PostgreSQL 18

SQL

Showing rows: 1 to 2 of 1

emp_id [PK] integer	emp_name character varying (50)	department character varying (50)	salary numeric (10,2)	city character varying (50)
1	105 Rahul	HR	48000.00	Chennai
2	109 Ravi	Marketing	42000.00	Chennai

Query

```
1 SELECT * FROM Employee
2 WHERE emp_name LIKE 'Ra%';
```

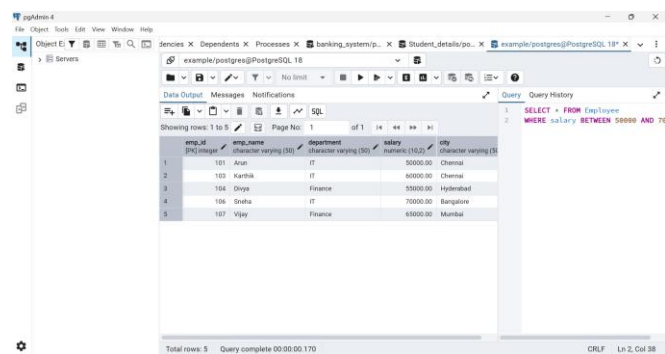
Successfully run. Total query runtime: 146 msec. 2 rows affected.

Total rows: 2 Query complete 00:00:00.146 CRLF Ln 2, Col 24

BETWEEN:

SELECT * FROM Employee

WHERE salary BETWEEN 50000 AND 70000;



The screenshot shows the pgAdmin 4 interface with a query window open. The query is: `SELECT * FROM Employee WHERE salary BETWEEN 50000 AND 70000;`. The result set displays 5 rows of employee data.

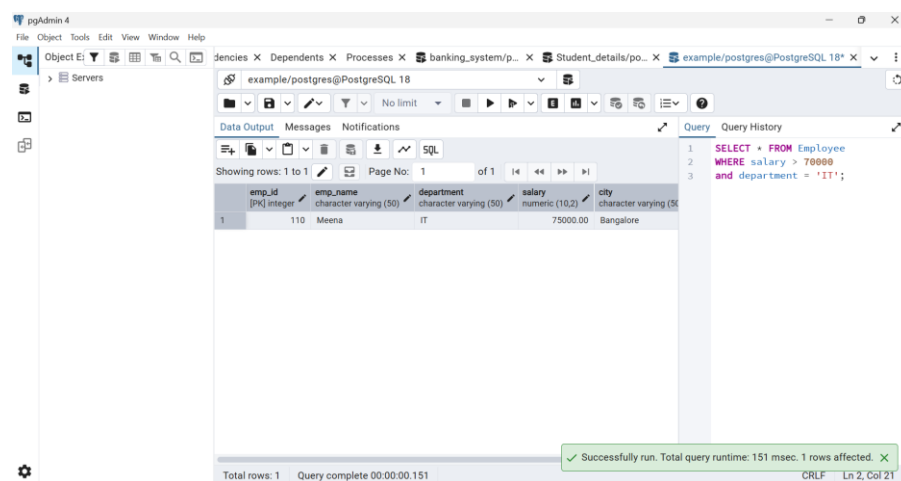
emp_id	emp_name	department	salary	city
101	Arun	IT	60000.00	Chennai
102	Karthik	IT	60000.00	Chennai
104	Dhruv	Finance	50000.00	Hyderabad
106	Snaha	IT	70000.00	Bangalore
107	Vijay	Finance	60000.00	Mumbai

AND:

SELECT * FROM Employee

WHERE salary > 70000

and department = 'IT';



The screenshot shows the pgAdmin 4 interface with a query window open. The query is: `SELECT * FROM Employee WHERE salary > 70000 and department = 'IT';`. The result set displays 1 row of employee data.

emp_id	emp_name	department	salary	city
110	Meena	IT	75000.00	Bangalore

Below the table, a green status bar indicates: "Successfully run. Total query runtime: 151 msec. 1 rows affected."

OR:

SELECT * FROM Employee

WHERE department = 'IT'

or

department = 'HR';

pgAdmin 4

example/postgres@PostgreSQL 18

Showing rows: 1 to 6 of 1

emp_id [PK] integer	emp_name character varying (50)	department character varying (50)	salary numeric (10,2)	city character varying (50)
1	101 Arun	IT	50000.00	Chennai
2	102 Priya	HR	45000.00	Bangalore
3	103 Karthik	IT	60000.00	Chennai
4	105 Rahul	HR	48000.00	Chennai
5	106 Sneha	IT	70000.00	Bangalore
6	110 Meena	IT	75000.00	Bangalore

Total rows: 6 Query complete 00:00:00.152 CRLF Ln 4, Col 6

```

1 SELECT * FROM Employee
2 WHERE department = 'IT'
3 or
4 department = 'HR';

```

IN:

SELECT * FROM Employee

WHERE department IN('IT','Finance');

pgAdmin 4

example/postgres@PostgreSQL 18

Showing rows: 1 to 6 of 1

emp_id [PK] integer	emp_name character varying (50)	department character varying (50)	salary numeric (10,2)	city character varying (50)
1	101 Arun	IT	50000.00	Chennai
2	103 Karthik	IT	60000.00	Chennai
3	104 Divya	Finance	55000.00	Hyderabad
4	106 Sneha	IT	70000.00	Bangalore
5	107 Vijay	Finance	65000.00	Mumbai
6	110 Meena	IT	75000.00	Bangalore

Total rows: 6 Query complete 00:00:00.157 CRLF Ln 2, Col 34

```

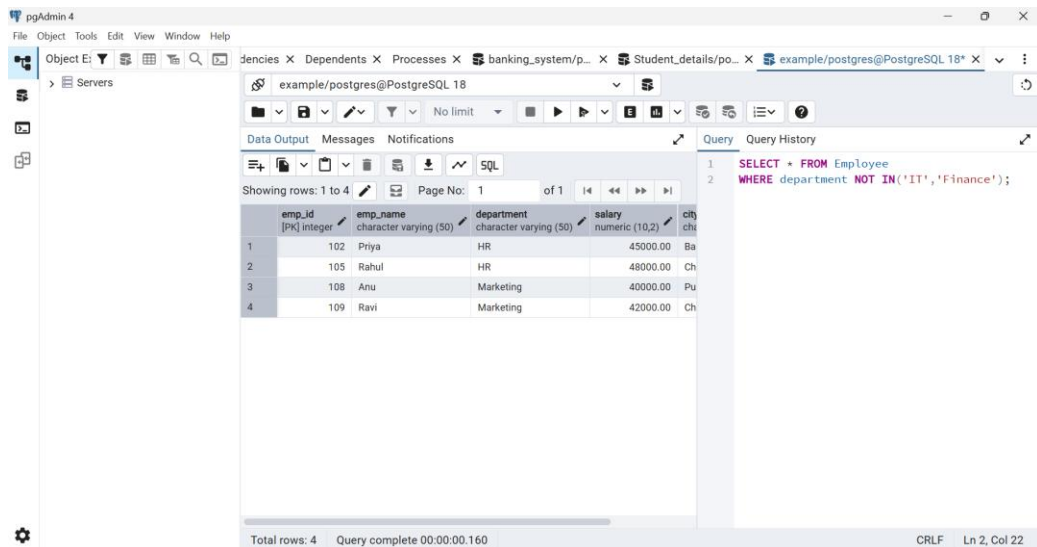
1 SELECT * FROM Employee
2 WHERE department IN('IT','Finance')

```

NOT IN:

SELECT * FROM Employee

WHERE department NOT IN('IT','Finance');



emp_id [PK] integer	emp_name character varying (50)	department character varying (50)	salary numeric (10,2)	city character varying (50)
102	Priya	HR	45000.00	Bangalore
105	Rahul	HR	48000.00	Chennai
108	Anu	Marketing	40000.00	Pune
109	Ravi	Marketing	42000.00	Chennai

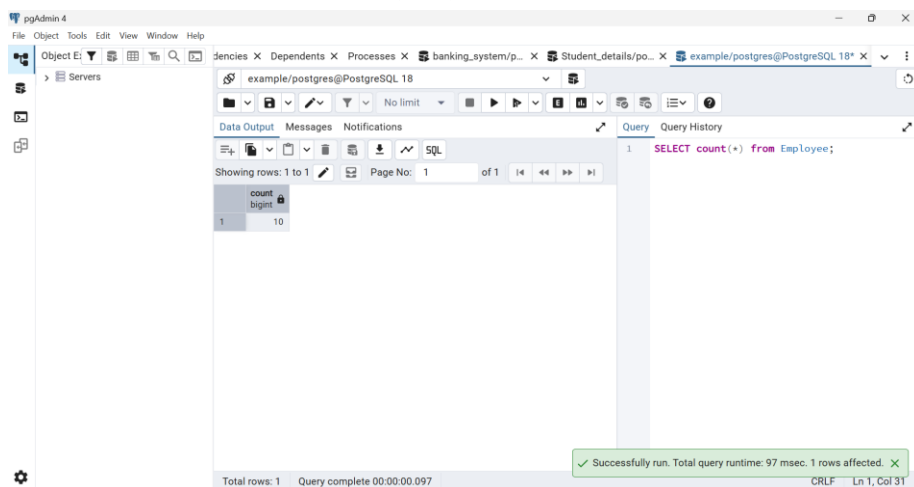
Total rows: 4 Query complete 00:00:00.160 CRLF Ln 2, Col 22

4. Aggregate Functions

Write queries using:

1. COUNT()

SELECT count(*) from Employee;



count bigint
10

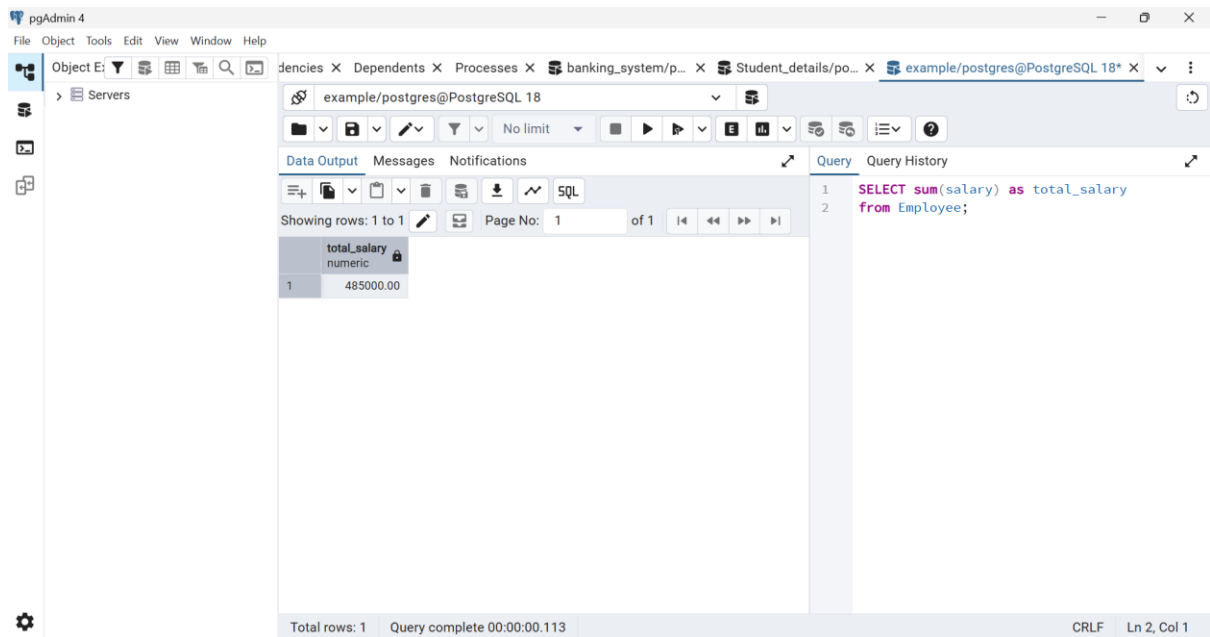
Total rows: 1 Query complete 00:00:00.097 CRLF Ln 1, Col 31

Successfully run. Total query runtime: 97 msec. 1 rows affected.

2.SUM()

SELECT sum(salary) as total_salary

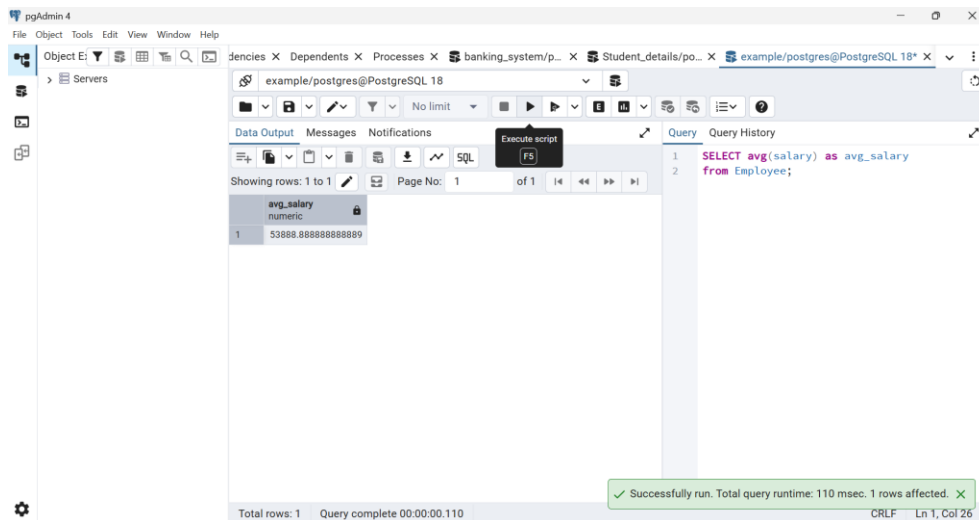
from Employee;



3.AVG()

SELECT avg(salary) as avg_salary

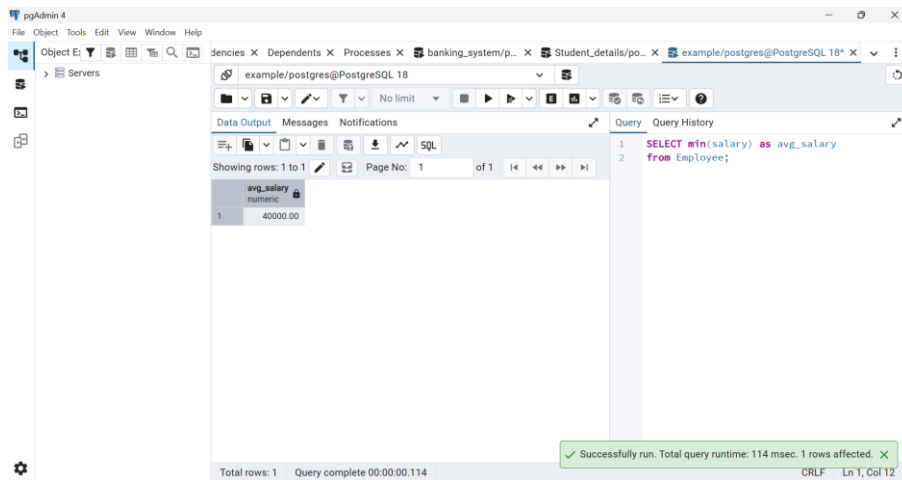
from Employee;



4.MIN()

SELECT min(salary) as avg_salary

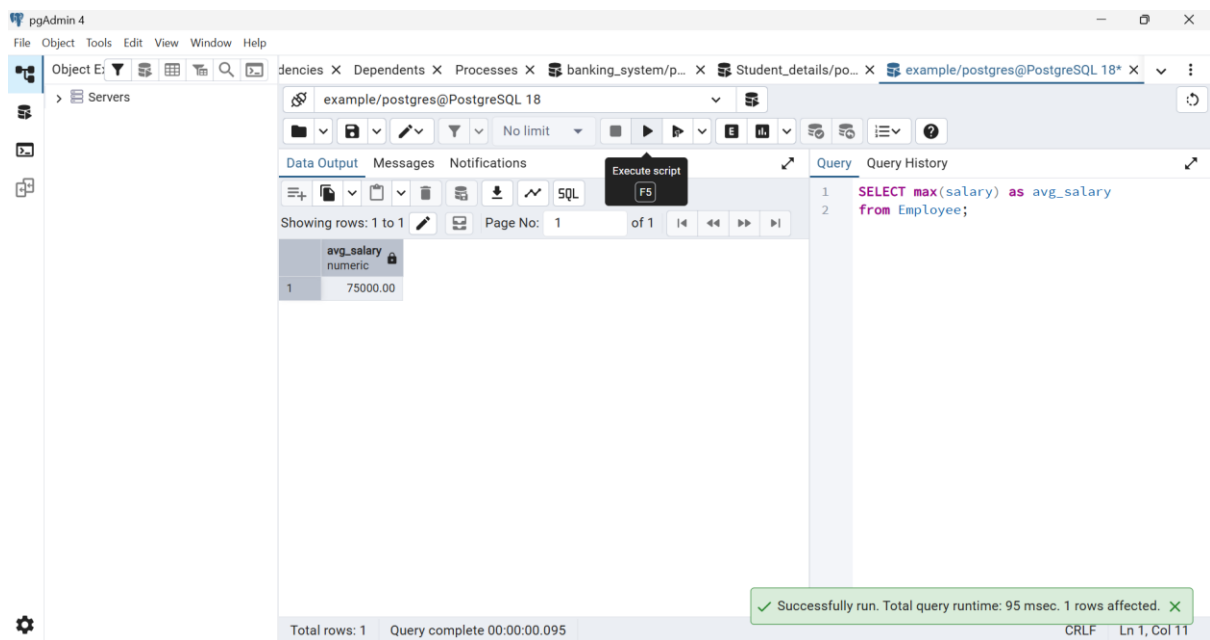
from Employee;



5.MAX()

SELECT max(salary) as avg_salary

from Employee;



5. GROUP BY & HAVING*

1) Count employees in each department.

SELECT department,count(*) as Employee_count

from Employee

group by department;

The screenshot shows the pgAdmin 4 interface with a query executed on a PostgreSQL database. The query is: `SELECT department, count(*) as Employee_count from Employee group by department;`. The result is displayed in a table with 4 rows.

	department character varying (50)	employee_count bigint
1	Marketing	2
2	Finance	1
3	IT	4
4	HR	2

Total rows: 4 Query complete 00:00:00.145 CRLF Ln 3, Col 10

2) Find the average salary for each department.

`SELECT department, avg(salary) as AVG_EACHDEPT from Employee group by department;`

The screenshot shows the pgAdmin 4 interface with a query executed on a PostgreSQL database. The query is: `SELECT department, avg(salary) as AVG_EACHDEPT from Employee group by department;`. The result is displayed in a table with 4 rows.

	department character varying (50)	avg_eachdept numeric
1	Marketing	41000.000000000000
2	Finance	55000.000000000000
3	IT	63750.000000000000
4	HR	46500.000000000000

Total rows: 4 Query complete 00:00:00.143

Successfully run. Total query runtime: 143 msec. 4 rows affected.

3) Find the total salary for each department.

`SELECT department, sum(salary) as Total_salary from Employee group by department;`

The screenshot shows the pgAdmin 4 interface. The SQL query editor contains the following query:

```
1 SELECT department, sum(salary) as Total_salary f
2  roup by department;
```

The Data Output pane shows the results of the query:

department	total_salary
Marketing	82000.00
Finance	55000.00
IT	255000.00
HR	92000.00

The status bar at the bottom indicates: Total rows: 4, Query complete 00:00:00.160, Successfully run. Total query runtime: 160 msec. 4 rows affected.

4) Display only departments whose average salary is greater than a specific value.

SELECT department,

avg(salary) as avg_salary_dep from Employee

group by department

having avg(salary) > 40000;

The screenshot shows the pgAdmin 4 interface. The SQL query editor contains the following query:

```
1 SELECT department,
2  avg(salary) as avg_salary_dep from Employee
3  group by department
4  having avg(salary) > 40000;
```

The Data Output pane shows the results of the query:

department	avg_salary_dep
Marketing	41000.000000000000
Finance	55000.000000000000
IT	63750.000000000000
HR	46500.000000000000

The status bar at the bottom indicates: Total rows: 4, Query complete 00:00:00.151.

6. SQL JOINS

Create *Employee* and *Department* tables and practice:

* INNER JOIN

SELECT e.emp_id,

e.emp_name,

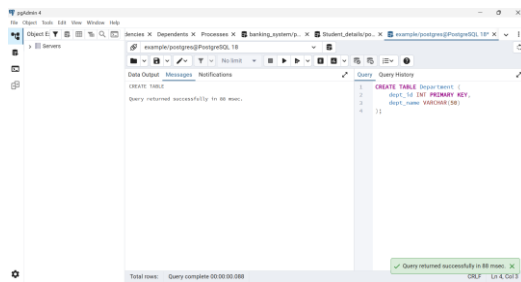
e.department,

d.location

FROM Employee e

INNER JOIN Department d

ON e.department = d.dept_name;



Returns only matching records from both tables.

* LEFT JOIN

Returns all employees and matching department details.

SELECT e.emp_id,

e.emp_name,

e.department,

d.location

FROM Employee e

LEFT JOIN Department d

ON e.department = d.dept_name;

pgAdmin 4

example/postgres@PostgreSQL 18

SQL

```

1 SELECT e.emp_id,
2       e.emp_name,
3       e.department,
4       d.dept_id
5 FROM Employee e
6 LEFT JOIN Department d
7 ON e.department = d.dept_name;

```

emp_id	emp_name	department	dept_id
110	Meena	IT	1
106	Sneha	IT	1
103	Karthik	IT	1
101	Arun	IT	1
105	Rahul	HR	2
102	Priya	HR	2
104	Divya	Finance	3
109	Ravi	Marketing	4
108	Anu	Marketing	4

Total rows: 9 Query complete 00:00:00.142

* RIGHT JOIN

SELECT e.emp_id,

e.emp_name,

e.department,

d.dept_id,

d.dept_name

FROM Employee e

RIGHT JOIN Department d

ON e.department = d.dept_name;

pgAdmin 4

example/postgres@PostgreSQL 18

SQL

```

1 SELECT e.emp_id,
2       e.emp_name,
3       e.department,
4       d.dept_id,
5       d.dept_name
6 FROM Employee e
7 RIGHT JOIN Department d
8 ON e.department = d.dept_name;

```

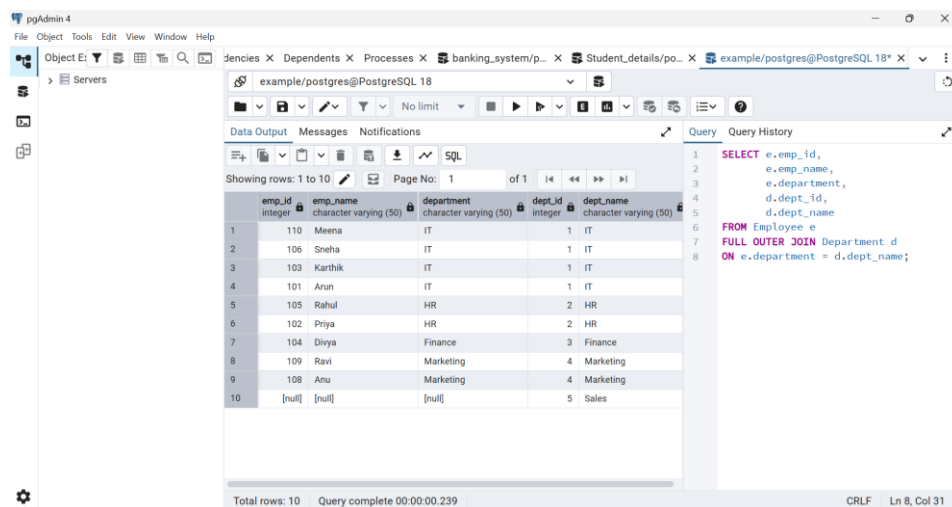
emp_id	emp_name	department	dept_id
110	Meena	IT	1
106	Sneha	IT	1
103	Karthik	IT	1
101	Arun	IT	1
105	Rahul	HR	2
102	Priya	HR	2
104	Divya	Finance	3
109	Ravi	Marketing	4
108	Anu	Marketing	4
[null]	[null]	[null]	5

Total rows: 10 Query complete 00:00:00.151

Successfully run. Total query runtime: 151 msec. 10 rows affected.

* FULL OUTER JOIN

```
SELECT e.emp_id,  
       e.emp_name,  
       e.department,  
       d.dept_id,  
       d.dept_name  
FROM Employee e  
FULL OUTER JOIN Department d  
ON e.department = d.dept_name;
```



The screenshot shows the pgAdmin 4 interface. The SQL query is entered in the 'Query' tab, and the results are displayed in the 'Data Output' tab. The query is a FULL OUTER JOIN between the Employee and Department tables. The results show 10 rows, including employees from different departments and departments without employees.

emp_id	emp_name	department	dept_id	dept_name
110	Meena	IT	1	IT
106	Sneha	IT	1	IT
103	Karthik	IT	1	IT
101	Arun	IT	1	IT
105	Rahul	HR	2	HR
102	Priya	HR	2	HR
104	Divya	Finance	3	Finance
109	Ravi	Marketing	4	Marketing
108	Anu	Marketing	4	Marketing
[null]	[null]	[null]	5	Sales

7. Customer & Orders*

Create *Customer* and *Orders* tables and write queries to:

```
CREATE TABLE Customer ( customer_id INT PRIMARY KEY,  
customer_name VARCHAR(50), city VARCHAR(50) );
```

```
INSERT INTO Customer VALUES
```

```
(1,'Arun','Chennai'),
```

```
(2,'Priya','Bangalore'),  
(3,'Rahul','Hyderabad'),  
(4,'Divya','Mumbai'),  
(5,'Sneha','Pune');
```

*** Display customer orders.**

```
SELECT c.customer_id,  
       c.customer_name,  
       o.order_id,  
       o.amount  
FROM Customer c  
INNER JOIN Orders o  
ON c.customer_id = o.customer_id;
```

*** Find customers without orders.**

```
SELECT c.customer_id,  
       c.customer_name  
FROM Customer c  
LEFT JOIN Orders o  
ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

*** Calculate the total purchase amount for each customer.**

```
SELECT c.customer_name,  
       SUM(o.amount) AS total_purchase  
FROM Customer c
```

```
INNER JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
* Find the customer who spent the most.
```

```
SELECT c.customer_name,
       SUM(o.amount) AS total_spent
FROM Customer c
INNER JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY total_spent DESC
LIMIT 1;
```

8. Advanced SQL Queries*

Write queries to:

*** Find the second highest salary.**

```
SELECT MAX(salary) AS second_highest_salary
FROM Employee
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee
);
```

*** Find the third highest salary.**

```
SELECT MAX(salary) AS third_highest_salary
```

```
FROM Employee
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee
    WHERE salary < (
        SELECT MAX(salary)
        FROM Employee
    )
);
```

*** Display the top 5 highest-paid employees.**

```
SELECT *
FROM Employee
ORDER BY salary DESC
LIMIT 5;
```

*** Find employees earning more than the average salary.**

9. Subqueries*

Practice writing subqueries to:

*** Find employees earning above the average salary.**

```
SELECT emp_name,
    salary
FROM Employee
WHERE salary > (
    SELECT AVG(salary)
```

```
FROM Employee  
);
```

*** Find the department with the highest average salary.**

```
SELECT department,  
       AVG(salary) AS avg_salary  
FROM Employee  
GROUP BY department  
ORDER BY avg_salary DESC  
LIMIT 1;
```

*** Compare employee salaries within departments.**

```
SELECT emp_name,  
       department,  
       salary  
FROM Employee e  
WHERE salary >  
(  
    SELECT AVG(salary)  
    FROM Employee  
    WHERE department = e.department  
);
```

10. Mini SQL Challenge

Solve the following:

*** Find the department with the highest average salary.**

```
SELECT department,  
       AVG(salary) AS avg_salary  
FROM Employee  
GROUP BY department  
ORDER BY avg_salary DESC  
LIMIT 1;
```

*** Find employees earning more than their department's average salary.**

```
SELECT emp_name,  
       department,  
       salary  
FROM Employee e  
WHERE salary >  
(  
    SELECT AVG(salary)  
    FROM Employee  
    WHERE department = e.department  
);
```

*** Find customers who placed more than two orders.**

```
SELECT emp_name,  
       department,  
       salary  
FROM Employee e  
WHERE salary >  
(
```

```
SELECT AVG(salary)
FROM Employee
WHERE department = e.department
);
```

*** Find products that have never been ordered.**

```
SELECT *
FROM Product
WHERE product_id NOT IN
(
    SELECT product_id
    FROM Order_Items
);
```

*** Find the second highest salary *without using LIMIT*.**

```
SELECT MAX(salary) AS second_highest_salary
FROM Employee
WHERE salary <
(
    SELECT MAX(salary)
    FROM Employee
);
```